01b Java

01b Java

00 Übersicht · Zurück: 01a Programmiersprachen · Weiter: 02 Variablen

Struktur objektorientierter Programme

```
Programm
  Klasse(n)
      Membervariablen
      Methoden
```

Element	Beschreibung
Programm	besteht aus **mindestens einer Klasse**. Programmname = Name der **Startklasse**; benennt üblicherweise die Aufgabe bzw. das Problem
Klasse	repräsentiert ein **Konzept** des Problems mit Verhalten und Zuständen. Jede Klasse steht in einer **eigenen Datei mit gleichem Namen**
Membervariablen	Instanzvariablen, Klassenvariablen, Konstanten. Beschreiben Eigenschaften einer Klasse; beliebig viele, auch keine
Methoden	beschreiben das **Verhalten**; berechnen etwas und/oder verändern Werte von Instanzvariablen; beliebig viele, auch keine

Beispiel Gauss

```java
class Gauss {
    // Membervariablen (hier leer)

    // Methoden
    public static void main(String[] args) {

        // Einlesen der Obergrenze von der Kommandozeile
        int boundary = Integer.parseInt(args[0]);

        // Berechnung der Summe nach Gauss
        int sum = boundary * (boundary + 1) / 2;
```

```
        // Alternative Berechnung der Summe
        // int sum = (n * n + n) / 2;

        // Ausgabe des Ergebnisses
        System.out.println(sum);
    }
}
```

- `main` ist der Einstiegspunkt; `args[0]` ist das erste Kommandozeilenargument.
- `Integer.parseInt` wandelt den String in einen `int`.
- Zu Entwurf und Phasen: 01 Einführung

Programmierumgebungen

Ablauf: `Gauss.java` (Programm) → **Compiler** → `Gauss.class` (Bytecode) → **Interpreter** → Ausgabe

	Variante 1 (Konsole)	Variante 2 (IDE, Standard im Kurs)
Schreiben	im Java-Editor	in der IDE
Übersetzen	`javac Gauss.java`	automatisch durch den Compiler der IDE
Ausführen	`java Gauss`	in der IDE (z.B. grüner Pfeil)
Ausgabe	Konsole (cmd, PowerShell, Terminal, bash)	Terminal in der IDE

IDE = *Integrated Development Environment*, z.B. IntelliJ, Visual Studio Code, NetBeans, Eclipse.

JDK und JRE

	JRE (Java Runtime Environment)	**JDK** (Java Development Kit)
Enthält	–	ein vollständiges JRE **plus** Entwicklungswerkzeuge (`javac`, `java`, `javadoc`, …)
Aufgaben	Schnittstelle zum Betriebssystem (simuliert dessen Dienste), Ablaufumgebung für den Interpreter (Bytecode), **Speicherverwaltung (Garbage Collector)**, Bibliotheken	Erkennbar am Kürzel „JDK“ im Namen der Download-Datei

> **Achtung**
> Zum Programmieren braucht man **immer beides** → daher immer das **JDK** herunterladen.

Geschichte und Versionen

Jahr	Ereignis
1995	erste Urversion von **Sun Microsystems**
1998	Java 1.2 (J2SE): erste ernstzunehmende Version
2004	Java 1.5 → Java 5: umfassende Überarbeitung und Erweiterung
2010	**Oracle** kauft Sun, entwickelt Java als quelloffene Software weiter
2014	Java 8: funktionale Aspekte
2018	Java 11: erstmals kostenpflichtig im produktiven Einsatz (Entwicklung/Test weiter frei), sehr häufige Versionswechsel

OpenJDK (openjdk.org): offizielle freie Implementierung der Java Plattform (Standard Edition), entstanden als Abspaltung von Java 6.
- aktuelle **LTS**-Version (Long Term Support): OpenJDK 25
- aktuelle Version: OpenJDK 27
- **Praktikum: OpenJDK 25 bis 27**

[01_Einführung.pdf, p.15]

> **Take-away**
> Viele grundlegende Konzepte findet man in anderen Sprachen wieder; der Unterschied liegt vor allem in der **Syntax**.